Water Sort! Color Sort Puzzle — Master Build Prompt for Replit Agent

One prompt for the whole game. Android · Expo / React Native · fully offline · monetized by ads only.

0. Your role and how to work

You are a senior mobile game engineer, game designer and UI artist. Build the complete game below from scratch as a Replit **Mobile app** (Expo + React Native + TypeScript), at production quality, ready for Google Play.

- Follow the phases in section 13, in order. At the end of each phase: run the app, fix every runtime and TypeScript error, run the tests, update `PROGRESS.md` (done / next / known issues) and `replit.md`, then continue straight to the next phase.
- Do not ask me questions. Choose sensible defaults and record each decision in `README.md`.
- No TODOs, lorem ipsum, fake data, dead buttons or stub screens: every visible control must work.
- Game logic is pure TypeScript (no React or Skia imports) and unit-tested. Keep files focused (under about 400 lines).
- Install packages with `npx expo install` so versions match the Expo SDK. Never invent package names or APIs; check types or docs when unsure.
- If budget runs low, stop only at a clean phase boundary and make `PROGRESS.md` precise enough to resume from.

1. Product

- Name: **Water Sort! Color Sort Puzzle** (29 characters; Google Play's title limit is 30). Keep the name and Android package id in one place (`src/config/brand.ts` and `app.config.ts`).
- Genre: classic liquid-sorting puzzle. Tubes hold layers of colored liquid; pour the top layer(s) onto the same color or into an empty tube until every color sits in a single tube. Calm and satisfying: no timers, no lives, no penalties.
- Business model: free, **ads only** — banner, app-open, interstitial and rewarded. No in-app purchases, subscriptions or remove-ads.
- Retention: endless levels, daily content, many kinds of rewards and a very large cosmetics store.

- Market baseline to match or beat: Undo / Restart / Hint / Add-tube boosters; thousands of levels; hidden `?` layers and special levels with long or mixed tubes; daily challenge and daily rewards; bottle, theme and background collections; soothing water sounds and music; full offline play. Player complaints to avoid: too many ads, rewarded ads that fail to pay out, lost progress.

2. Hard constraints

1. Android only, portrait only, phones and tablets. No web or iOS work. Use the latest stable Expo SDK and make sure the final Android build targets API level 36 or higher (Google Play's requirement for new apps). Edge-to-edge is mandatory: handle safe-area insets on every screen and Android back / predictive back (back closes the top modal first; on Home it asks to exit).
2. 100% offline. The only network traffic allowed is Google Mobile Ads and its consent (UMP) form. No backend, accounts, analytics or crash SDKs, remote config, or remote images, fonts or audio.
3. No billing libraries or IAP code paths.
4. English-only UI, but every string lives in `src/i18n/en.ts`.
5. Original assets only: art drawn in code (Skia) or generated by scripts, audio synthesized by script. Nothing copied from other games or the web.
6. Android permissions: INTERNET, ACCESS_NETWORK_STATE, VIBRATE and `com.google.android.gms.permission.AD_ID` — nothing else.
7. Do not install `expo-dev-client` (it breaks the Expo Go preview in Replit). Native-only code (AdMob) sits behind an adapter that falls back to mock ads inside Expo Go (section 8).

3. Stack and structure

- Expo + React Native + TypeScript (`strict`) + expo-router (or the template's navigation).
- Rendering: `@shopify/react-native-skia` for the whole board and all effects; `react-native-reanimated` shared values drive Skia (no per-frame React re-renders); `react-native-gesture-handler` for taps.
- State: Zustand persisted to `@react-native-async-storage/async-storage`.
- `expo-audio` (maintained replacement for expo-av), `expo-haptics`, `expo-keep-awake`, `@expo/vector-icons`, and one rounded display font (e.g. Fredoka from `@expo-google-fonts`) bundled locally.

- Ads: `react-native-google-mobile-ads` with its Expo config plugin. After installing, confirm the plugin resolves (`npx expo config --type prebuild`); if the version mismatches the SDK, use the one `npx expo install --check` recommends. Never remove ads to dodge an error.
- Tests: Jest (jest-expo or ts-jest) for all pure logic.
- Layout:

```
app/            routes: index (Home), game, levels, store, daily-challenge,
daily-reward, spin, quests, achievements, settings
src/game/       types, rules, solver, generator, difficulty, hint, stars
(pure TS)
src/render/     Skia: board layout, Tube, Liquid, PourStream, Particles, Cap,
Background
src/state/      stores, persistence, migrations, clock
src/services/   ads/ (AdService, AdMobProvider, MockProvider, consent,
policy), audio/, haptics/
src/data/       levels.fixed.json, catalog/, achievements, quests,
dailyRewards, wheel
src/ui/         theme and shared components (Button, Modal, Toast,
CurrencyBar, Chest...)
src/config/     brand, ads, economy, difficulty
scripts/        generate-levels, validate-levels, generate-audio, generate-
icons
docs/           ADS_SETUP, PUBLISHING_CHECKLIST, PRIVACY_POLICY,
STORE_LISTING, AUDIO_CREDITS
```

4. Core gameplay

- Model: `Tube { capacity, layers[] (bottom to top) }`, `Layer { color: number, hidden: boolean }`. The color index maps to a real color through the equipped Liquid theme.
- Controls (one finger): tap a tube to select it (it lifts and glows), tap another tube to pour. Tapping the same tube cancels. Tapping a different non-empty tube that cannot receive the pour moves the selection to it. Tapping an invalid empty or full tube makes it shake with a soft error sound. Input is locked only during a pour animation (under 900 ms).
- Pour rule: the source is not empty, the target is not full, and the target is empty or its top color equals the source's top color. The whole contiguous top run of that color moves, limited by the target's free space.
- Hidden layers show `?` until they become the top layer, then reveal with a small pop animation and sound.

- Complete tube: a tube holding every unit of one color closes with a cap (spring drop), sparkles and a chime whose pitch rises with each consecutive completion; it can no longer be selected.
- Win check (generic): every tube is empty or complete, i.e. each tube is single-color and no color is split across tubes. It must support mixed capacities and uneven unit counts.
- Undo: reverts the last pour exactly (re-hiding any layer it revealed); unlimited history within a level; costs 1 Undo token. Restart: free and unlimited; asks for confirmation after 5 or more moves.
- Hint: costs 1 Hint token; runs the solver and highlights source, then target, with an animated arrow. If the position is unsolvable, say so for free and suggest Undo or Restart.
- Add Tube: adds one empty tube of the level's standard capacity, max 1 per level (2 on Hard Levels), costs 1 Tube token; animated slide-in and layout reflow.
- Stuck detection: no legal moves, or the solver proves a dead end → a non-blocking sheet offers Undo, Add Tube, Restart and Skip (via rewarded ad).
- Skip level: rewarded ad, offered in the pause menu after the 2nd restart or 3 minutes on a level; counts as cleared with 1 star and no coins.
- Stars: par = the solver's best known move count. 3 stars at 1.5× par or less, 2 stars at 2.5× par or less, otherwise 1. Undone pours do not count.

5. Levels, generator and solver

- Fixed campaign: levels 1–300 are generated offline by `scripts/generate-levels.ts`, saved as compact strings in `src/data/levels.fixed.json` and re-checked by `scripts/validate-levels.ts` (solvable, inside the difficulty band, no duplicates).
- Endless: level 301+ is generated at runtime, deterministically from the level number (seeded PRNG). Pre-generate the next level in the background in small time slices (no more than 8 ms per frame) so the UI never janks. If verification times out, reroll with the next seed.
- Daily Challenge: generated from the date seed (same puzzle for everyone, no server), always with hidden layers.
- Pipeline: start from a solved board → scramble with reverse pours that are exact inverses of legal forward pours (solvable by construction) → reject trivial boards (tubes already sorted at the start, boards a greedy strategy solves) → **verify with the forward solver** (canonical-state hashing, memoized DFS or A*, time- and node-boxed) → score difficulty → accept only boards inside the target band for that level range. Never ship or serve an unverified level. Target feel: fair but a bit hard.

- The same solver powers Hint, par/stars and dead-end detection. It runs in time-sliced async chunks and never blocks the UI.
- Difficulty ramp (tubes = colors + empty tubes):
  - 1–5 tutorial: capacity 4, 2–3 colors, 1–2 empty tubes.
  - 6–20: capacity 4, 4–6 colors, 2 empty.
  - 21–60: capacity 4, 6–9 colors, 2 empty.
  - 61–120: capacity 4 → 5, 8–11 colors, 2 empty; hidden layers from level 80 (10–30% of non-top layers).
  - 121–300: capacity 5–6, 10–14 colors, 2 empty (1 on some levels); hidden layers 20–50%; a special level with mixed tube sizes roughly every 7th level.
  - 301+: capacity 5–8 (never above ~100 layers or 18 tubes), up to 16 colors, hidden layers and mixed sizes blended in; difficulty plateaus at hard, with variety.
  - Every 10th level is a **Hard Level**: a label, double rewards, one extra allowed tube.
  - Booster tooltips unlock gradually: Undo at level 2, Hint at 4, Add Tube at 6.
- Layout: choose 1–3 rows (up to 18 tubes) that give the largest tubes; scale uniformly between the HUD and the banner; minimum tap target 44 dp; must work from 320 dp phones to tablets.
- Levels screen: pages of 50 with stars; cleared levels can be replayed to improve stars (no coins on replays).

6. Economy and rewards

Currencies: **Coins** (common) and **Gems** (rare), earned only by playing and by watching rewarded ads. Tokens: Undo, Hint, Tube. Starting inventory: 10 Undo, 5 Hint, 3 Tube. Every number lives in `src/config/economy.ts`.

- Level win: coins = 10 + min(level / 10, 20) + star bonus (2 stars +5, 3 stars +10); Hard Levels ×2. A rewarded ad on the win screen doubles the coins.
- Chests: level chest every 5 levels (40–80 coins + 1 random token, 10% chance of 3 gems); star chest every 30 total stars; free gift chest every 4 hours (30–60 coins, visible timer).
- Out of tokens: a bottom sheet offers Watch ad (+3 Undo / +2 Hint / +1 Tube) or spend coins. The store's Boosters tab sells bundles for coins.
- Welcome gift on first launch.
- Daily Reward: 7-day streak calendar (D1 100 coins, D2 2 Hints, D3 200 coins, D4 3 Undo + 1 Tube, D5 300 coins, D6 5 Gems, D7 500 coins + 20 Gems + 2 Hints + 1 Tube). Missing a day resets the streak. Claim ×2 with a rewarded ad (coins and tokens only).

- Daily Spin: 8-segment wheel; one free spin per day plus up to 5 more via rewarded ads. Odds in %: 50 coins 30, 100 coins 22, 200 coins 10, 3 Undo 12, 2 Hints 12, 1 Tube 8, 5 Gems 5, jackpot 25 Gems 1. An info button shows the odds.
- Daily Challenge: monthly calendar; play today's puzzle; missed days of the current month can be replayed via rewarded ad. Chests at 7 / 15 / 25 completed days and a trophy badge for finishing the whole month.
- Daily Quests: 3 per day from templates (win N levels, earn 3 stars, win without Undo, make N pours, clear a hidden-layer level, watch 1 rewarded ad), 20 points each; the 20 / 40 / 60 point milestones open chests.
- Achievements: at least 40, each with bronze, silver and gold tiers and claimable rewards.
- Free coins: a button that gives 50 coins per rewarded ad (2-minute cooldown, max 10 per day).
- Time safety: store `lastSeenAt`; if the device clock moves back more than 5 minutes, pause every time-gated reward and show 'Check your device time'. Use local calendar-day keys (`YYYY-MM-DD`), never 24-hour deltas.

7. Store and skins

- Tabs: Tubes · Liquids · Backgrounds · Effects · Caps · Boosters · Free.
- Launch catalog of at least 120 items, fully data-driven in `src/data/catalog/*.ts` (adding an item means adding an object, no code change):
  - Tubes, at least 36: classic glass, crystal, neon, wood, gold, ice, stone, lava, candy, potion flask, beaker, jar, bamboo, steampunk, cyberpunk and more.
  - Liquids, at least 24 palettes of 16 colors each (Classic, Pastel, Neon, Candy, Ocean, Forest, Sunset, Galaxy, Retro, Earth, Jewel, Aurora…) with finishes (water, juice, jelly, glitter, neon glow), including 3 color-blind-safe palettes.
  - Backgrounds, at least 36: gradients plus procedural patterns (waves, bokeh, stars, hex grid, sunrays, wood, marble…).
  - Effects, at least 12: pour sparkle, bubble burst, star trail, confetti styles…
  - Caps, at least 14: cork, gold lid, crown, glass stopper…
- Everything is rendered procedurally in Skia (no image packs); store tiles show live previews.
- Rarity: Common, Rare, Epic, Legendary, with colored frames.
- Unlock types: coins, gems, ad progress (watch N rewarded ads, shown as 2/5), achievement, daily-streak day 7, level milestone, quest chest, daily-challenge trophy. At least 20% of items are earn-only.

- Pricing guide: Common 150–400 coins; Rare 500–1,200; Epic 1,500–4,000 coins or 40–100 gems; Legendary 150–300 gems or a special unlock. An average player (about 10 levels a day plus dailies) should afford a rare item every ~2 days and an epic about once a week.
- UX: Preview applies the item to a demo board, Equip applies instantly in-game, NEW badges, owned and equipped state persisted.

8. Ads (AdMob): four formats

- `AdService` interface: `init()`, `showAppOpenIfEligible()`, `showInterstitialIfEligible(reason)`, `showRewarded(placement): Promise<{ rewarded: boolean }>`, `isRewardedReady()`, plus a `BannerSlot` component.
- Two providers: `AdMobProvider` (real; loaded with a guarded `require`, only in standalone builds) and `MockProvider` (Expo Go, web, tests: a fake 'Test banner' strip and fake full-screen ads with a 3–5 s countdown and a close button, so every ad flow is testable in the Replit preview). Detect Expo Go with `expo-constants` (`executionEnvironment`) and also wrap the native `require` in try/catch. The app must never crash because the native module is missing.
- IDs live only in `src/config/ads.ts`: Android app id plus four unit ids (banner, app-open, interstitial, rewarded). Defaults are Google's official test values (`TestIds.*` and the sample app id `ca-app-pub-3940256099942544~3347511713`). A `USE_TEST_ADS` flag defaults to true and becomes false only for the EAS `production` profile (read `EAS_BUILD_PROFILE` in `app.config.ts` and expose it through `extra`). `app.config.ts` injects the app id into the plugin config. Write `docs/ADS_SETUP.md` explaining where to create the AdMob app and the four ad units and where to paste the ids. Never use live unit ids while testing.
- Consent: on every cold start run the UMP flow (request consent info → show the form if required → only then initialize the SDK). Show a 'Privacy settings' row in Settings when the library says privacy options are required. Request configuration: `maxAdContentRating = PG`, `tagForChildDirectedTreatment = false`, `tagForUnderAgeOfConsent = false`. If consent or ads are unavailable, the game continues normally without ads.
- Banner: anchored adaptive, at the bottom of Home, Levels, Store and Game (hidden on levels 1–2). Never on modals or over gameplay. Reserve its height so the layout never jumps and keep at least 16 dp between it and any tappable control.
- App-open: when returning to the foreground after at least 60 s in the background; at least 4 minutes since any other full-screen ad; never in the first session or before level 4; never during a pour or while a modal is open; ignore foreground events caused by our own ads; discard cached ads older than 4 hours.

- Interstitial: only at natural breaks, when the player taps Next on the level-complete screen. From level 5, every 2nd completed level, at least 90 s since the last full-screen ad. Never after Restart, Undo or Hint, never in the tutorial, and skip it if a rewarded ad was watched in the last 3 minutes.
- Rewarded: always player-initiated with a clear 'Watch ad → reward' button. Placements: double level coins, double daily reward, get more Undo/Hint/Tube, skip level, extra spin, free coins, replay a missed daily-challenge day, ad-progress skin unlocks. The reward is granted only on the SDK's earned-reward event. If no ad is ready, show a friendly toast. Preload the next ad immediately, retry failed loads with exponential backoff (cap 5 min) and persist per-day counters.
- Policy hygiene: never encourage clicking ads, never place ads where a tap could be accidental, never block the game while an ad loads (at most a 2 s spinner), and keep the AD_ID permission in the final manifest.
- Every threshold is a constant in `ads.ts` and is covered by unit tests with a fake clock.

9. Persistence, time and reliability

- One versioned `SaveData` object (coins, gems, tokens, progress and stars, settings, owned and equipped items, daily / quests / achievements state, ad counters, tutorial flags, stats, `lastSeenAt`) in AsyncStorage key `save_v1`, with a rolling backup key and a migration function per version. If the main save fails to parse, restore the backup and tell the player.
- Autosave debounced to 500 ms, plus immediately on every economy change, on level completion and when the app goes to the background. Persist the in-progress level (board, move history, boosters used) so the player resumes exactly after the app is killed.
- Leave Android Auto Backup enabled so progress can follow the player to a new phone without any server of our own.
- Settings → Reset progress wipes everything after a double confirmation.
- Debug panel, only when `__DEV__`: add coins, gems and tokens, jump to a level, shift the simulated date, toggle mock ads, reset save, force-complete the level.

10. Audio

- Generate every sound with `scripts/generate-audio.ts` (pure synthesis in Node, no downloads). Keep total audio under 10 MB (use compressed formats where possible) and make loops seamless.

- SFX: tap/select, deselect, illegal move, pour (3 variants, pitch rising as the target fills), hidden-layer reveal, tube complete (chime scale), cap close, level-complete fanfare, star pops (3 pitches), coin collect and count-up tick, chest open, spin tick and win, button click, toggle, popup open/close, reward claimed, hint sparkle, undo whoosh, tube slide-in, error / no-ad, achievement unlocked, daily claim.
- Music: 4 calm ambient loops rotating between levels; ducked while ads play. Separate Music and SFX sliders plus a mute switch. Pause when the app goes to the background and resume on return.
- List every file in `docs/AUDIO_CREDITS.md` as 'generated in-house' so the owner can replace files under the same names.

11. Visuals, screens and accessibility

- Look: realistic 2D. Glass tubes with transparency, rim thickness, glossy highlights, soft contact shadows on a surface and refraction lines at layer boundaries; liquids with vertical gradients and a glossy top surface whose wave animates and decays after a pour; a few rising bubbles; a curved pour stream; droplet splashes and ripples on impact.
- Pour animation: the source tube lifts and tilts toward the target (angle depends on its fill level), liquid levels change smoothly in both tubes, then the source returns. Duration scales with the number of layers (400–900 ms).
- Polish: spring easing, button press scale, confetti and star animation on win with a coin count-up, haptics for select / pour / complete (toggle in Settings). Design system on an 8 dp grid with rounded cards, consistent icons and one rounded font.
- Screens: Splash → consent → Home (Play with the current level, Daily Challenge, Daily Reward, Spin, Store, Quests, Achievements, Levels, Settings, currency bar, banner) → Game (HUD with level and currencies, pause, Restart, Undo, Hint, Add Tube with token badges, banner) → modals (win, pause, stuck, get-more, confirm). Settings: music, SFX, haptics, color-blind mode, graphics quality, privacy settings, privacy policy text bundled in the app, rate us via the Play Store link, share, how to play, reset, version.
- Tutorial: guided hand pointer on levels 1–3; each booster gets a one-time tooltip.
- Accessibility: color-blind mode overlays distinct white symbols or patterns on every liquid color; never rely on color alone; TalkBack labels on every control and a spoken summary per tube; honor the system reduce-motion setting by disabling shake and particles.
- Performance: 60 fps on mid-range phones and at least 30 fps on low-end; a Low graphics option removes waves, bubbles and particles; particle cap 120; no allocations in worklets and no per-frame JS-thread work.

12. Testing and quality gates

- Jest tests for: pour rules, hidden-layer reveal, win detection, undo, solver correctness on known puzzles, generator (at least 500 seeded boards across all ranges are solvable and inside their band), stars, economy math, daily-reward streaks and spin odds, ad-eligibility rules with a fake clock, save migrations and backup restore.
- `scripts/validate-levels.ts` passes on all 300 fixed levels.
- At the end of every phase: `tsc --noEmit`, lint and the full test suite pass, `npx expo-doctor` is clean, and a repo-wide search shows no network calls other than the ad SDK (no fetch, axios or XMLHttpRequest).

13. Build phases

1. **Foundation**: Expo project, dependencies, folder structure, theme, navigation, i18n, config files, Jest, `PROGRESS.md`.
2. **Engine**: rules, undo, win check, solver, hint, stars, generator, difficulty config, fixed-level script and JSON, validation script, unit tests.
3. **Board and feel**: Skia board, tubes, liquids, pour animation, particles, caps, backgrounds, selection logic, layout, haptics. Playable with the fixed levels.
4. **Meta systems**: persistence, economy, tokens, win and stuck flows, stars, Levels screen, tutorial, daily reward, spin, daily challenge, quests, achievements, chests, clock safety.
5. **Store and skins**: full catalog, previews, equip logic, unlock rules, ad-progress unlocks.
6. **Audio**: generator script, audio manager, settings.
7. **Ads**: AdService, both providers, consent, banner, app-open, interstitial, rewarded and every reward hook, policy rules, tests.
8. **Polish and release**: accessibility, Low graphics, performance pass, icons and splash, store assets, docs, `eas.json`, final QA.

A phase is done only when it runs in Expo Go, its tests pass and nothing in it is stubbed.

14. Release package

- `app.config.ts`: name, slug, version 1.0.0, `android.package` as one clearly marked placeholder (for example `com.yourstudio.watersortpuzzle`; it cannot be changed after publishing), versionCode 1, portrait, adaptive icon, splash, edge-to-edge, minimal permissions, the ads plugin, and `compileSdk` / `targetSdk` 36 or higher through `expo-build-properties` if needed.

- `eas.json`: `preview` (APK, internal distribution, test ads) and `production` (AAB, auto-incrementing versionCode, live ads) profiles. Do not run live-ad builds until I have added my ids.
- `scripts/generate-icons.ts` draws the 1024 app icon, adaptive icon layers, splash, 512 Play icon and a 1024×500 feature graphic from code.
- Docs: `README.md` (run, test, build, where to change the package name and ad ids); `docs/ADS_SETUP.md`; `docs/PUBLISHING_CHECKLIST.md` (Play Console app creation, store listing, content rating, target audience, Data safety, ads and advertising-ID declarations, privacy policy URL, the closed-testing requirement for new personal accounts, uploading the `.aab`, raising versionCode on every upload, Google-managed signing); `docs/PRIVACY_POLICY.md` (template covering AdMob, UMP and on-device data, with placeholders for contact details); `docs/STORE_LISTING.md` (title, short description under 80 characters, full description, tags, category Games › Puzzle).
- Final message to me: what was built, what is mocked in the preview, and the exact steps to paste my AdMob ids, change the package name, make a preview APK, make the production AAB and upload it.

15. Definition of done

- Level 1 through the endless range is playable end to end with every feature working: boosters, stars, rewards, store, dailies, spin, quests, achievements, settings.
- All four ad formats work through the mock provider in Expo Go and are wired to the real SDK for standalone builds, with test ids by default.
- Offline everywhere except ads; progress survives a kill and restart; corrupt saves recover.
- No TypeScript errors, passing tests, clean `expo-doctor`, smooth 60 fps feel, nothing stubbed, no placeholder text.